

SYSTEM CALLS AND TYPES



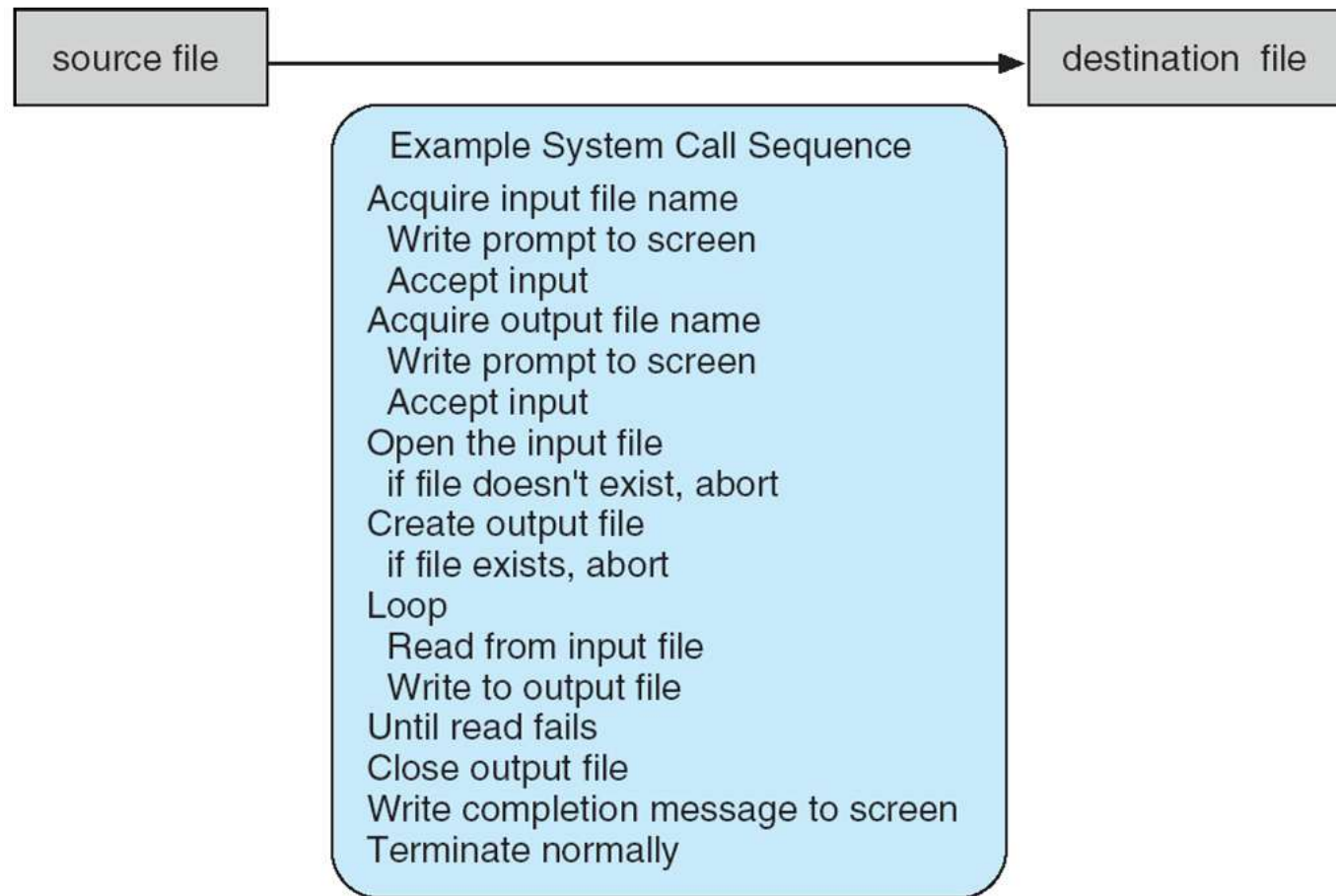
SYSTEM CALLS

- **System calls** provide an interface to the services made available by an operating system.
- Typically **written in a high-level language** (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use.
- Three most common APIs are **Win32 API for Windows**, **POSIX API for POSIX-based systems** (including virtually all versions of UNIX, Linux, and Mac OS X), and **Java API for the Java virtual machine (JVM)**



EXAMPLE OF SYSTEM CALLS

- System call sequence to copy the contents of one file to another file



EXAMPLE OF STANDARD API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t  read(int fd, void *buf, size_t count)
```

return value	function name	parameters
-----------------	------------------	------------

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

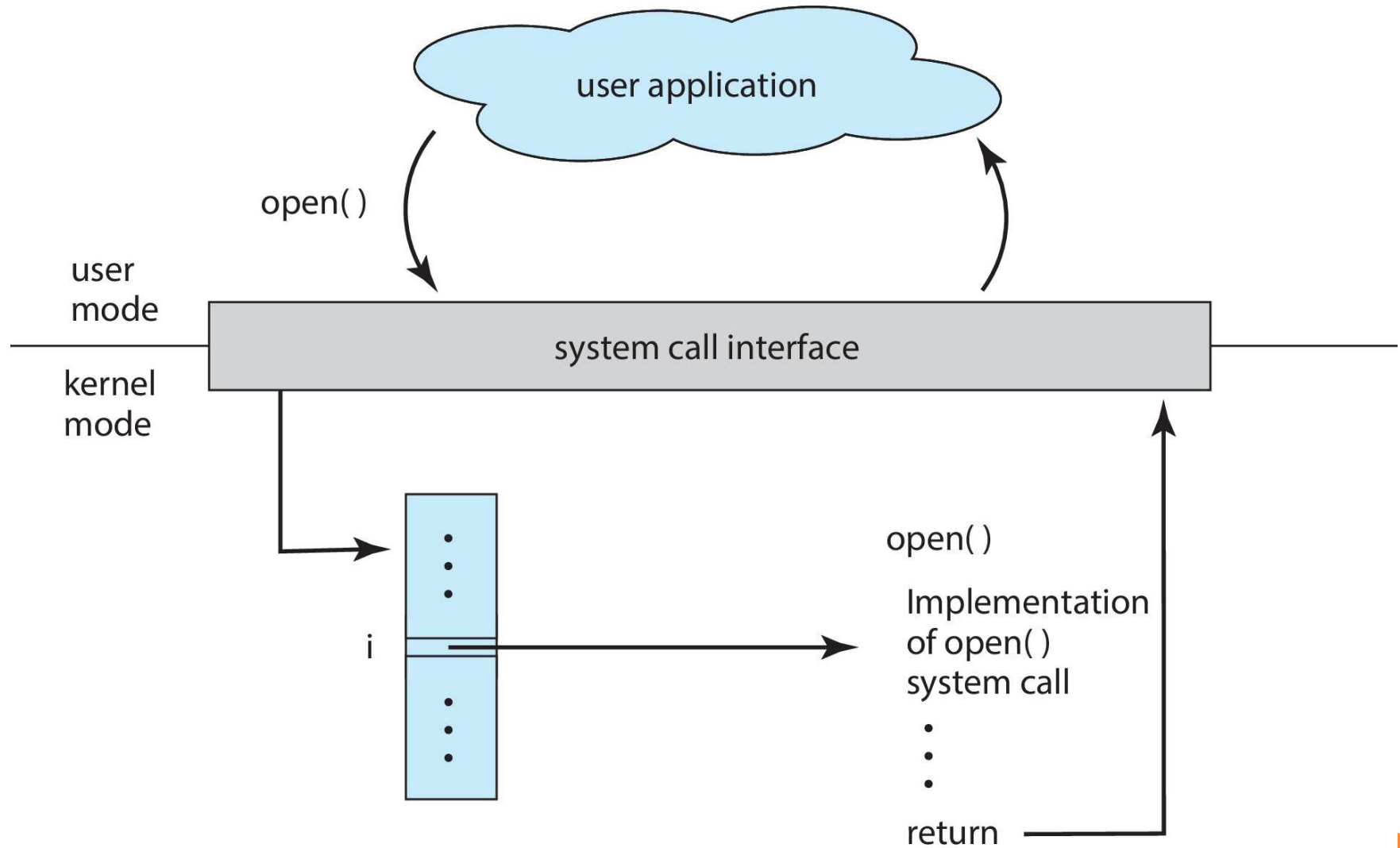


SYSTEM CALL IMPLEMENTATION

- Typically, a **number is associated with each system call**
 - **System-call interface** maintains a table indexed according to these numbers
- The **system call interface invokes** the intended **system call** in **OS kernel** and **returns status** of the system call and any return values
- The **caller needs to know nothing** about how the system call is implemented
 - Just needs to **obey API** and **understand what OS** will do as a result call
 - Most details of OS interface hidden from programmer by API
 - Managed by run-time support library (set of functions built into libraries included with compiler)



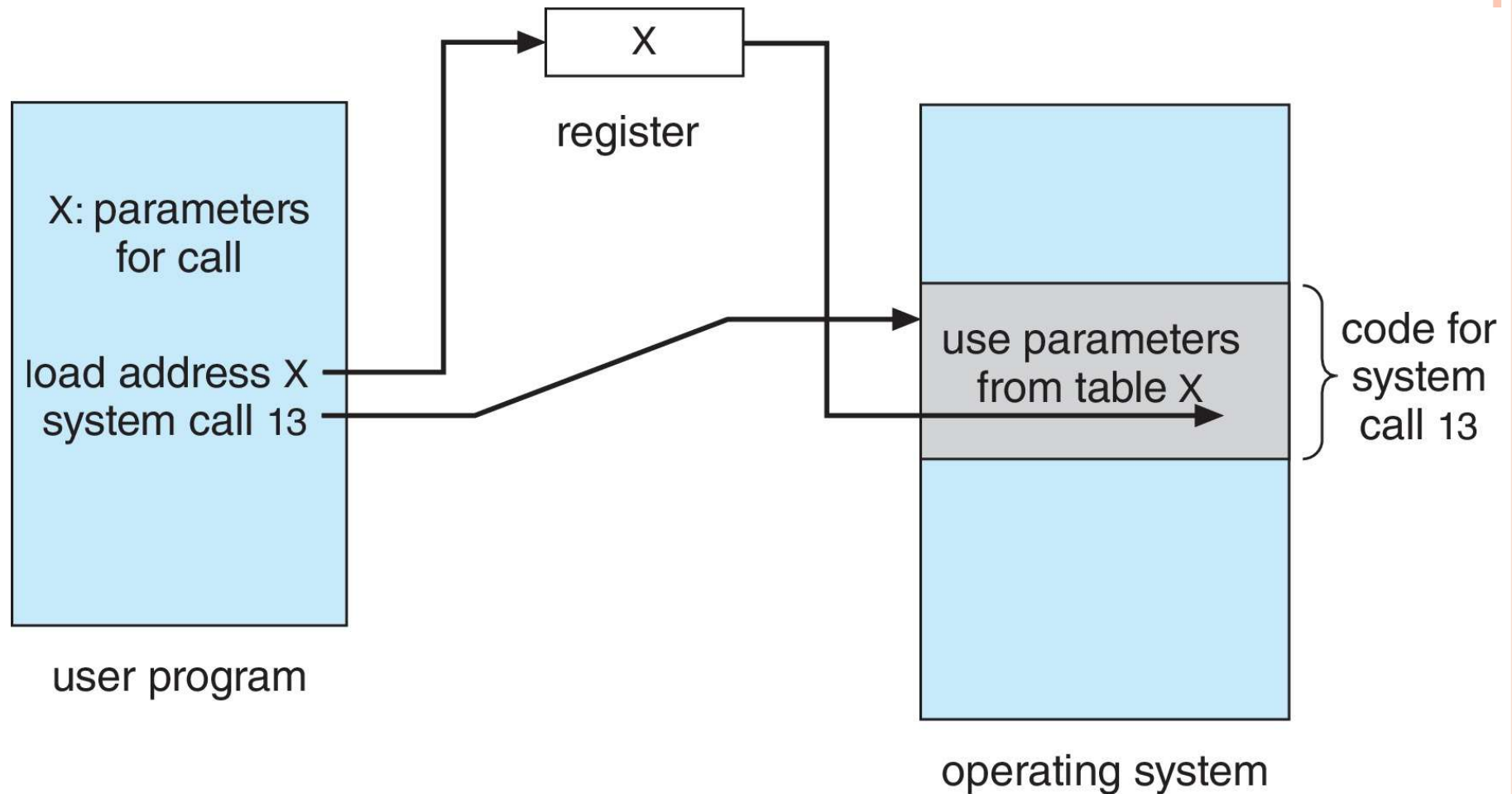
API – SYSTEM CALL – OS RELATIONSHIP



SYSTEM CALL PARAMETER PASSING

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call
- **Three general methods used to pass parameters to the OS**
 - **Simplest:** pass the parameters in registers
 - In some cases, may be more parameters than registers
 - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - This approach taken by Linux and Solaris
 - Parameters placed, or pushed, onto the stack by the program and popped off the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed

PARAMETER PASSING VIA TABLE



TYPES OF SYSTEM CALLS

- System calls can be grouped roughly into six major categories:
 1. **Process control,**
 2. **File management,**
 3. **Device management,**
 4. **Information maintenance,**
 5. **Communications,**
 6. **Protection**



TYPES OF SYSTEM CALLS

○ Process control

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory
- Dump memory if error
- **Debugger** for determining **bugs, single step** execution
- **Locks** for managing access to shared data between processes



TYPES OF SYSTEM CALLS (CONT.)

○ File management

- create file, delete file
- open, close file
- read, write, reposition
- get and set file attributes

○ Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices



TYPES OF SYSTEM CALLS (CONT.)

○ Information maintenance

- get time or date, set time or date
- get system data, set system data
- get and set process, file, or device attributes

○ Communications

- create, delete communication connection
- send, receive messages if **message passing model** to **host name** or **process name**
 - From **client** to **server**
- **Shared-memory model** create and gain access to memory regions
- transfer status information
- attach and detach remote devices



TYPES OF SYSTEM CALLS (CONT.)

○Protection

- Control access to resources
- Get and set permissions
- Allow and deny user access



EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

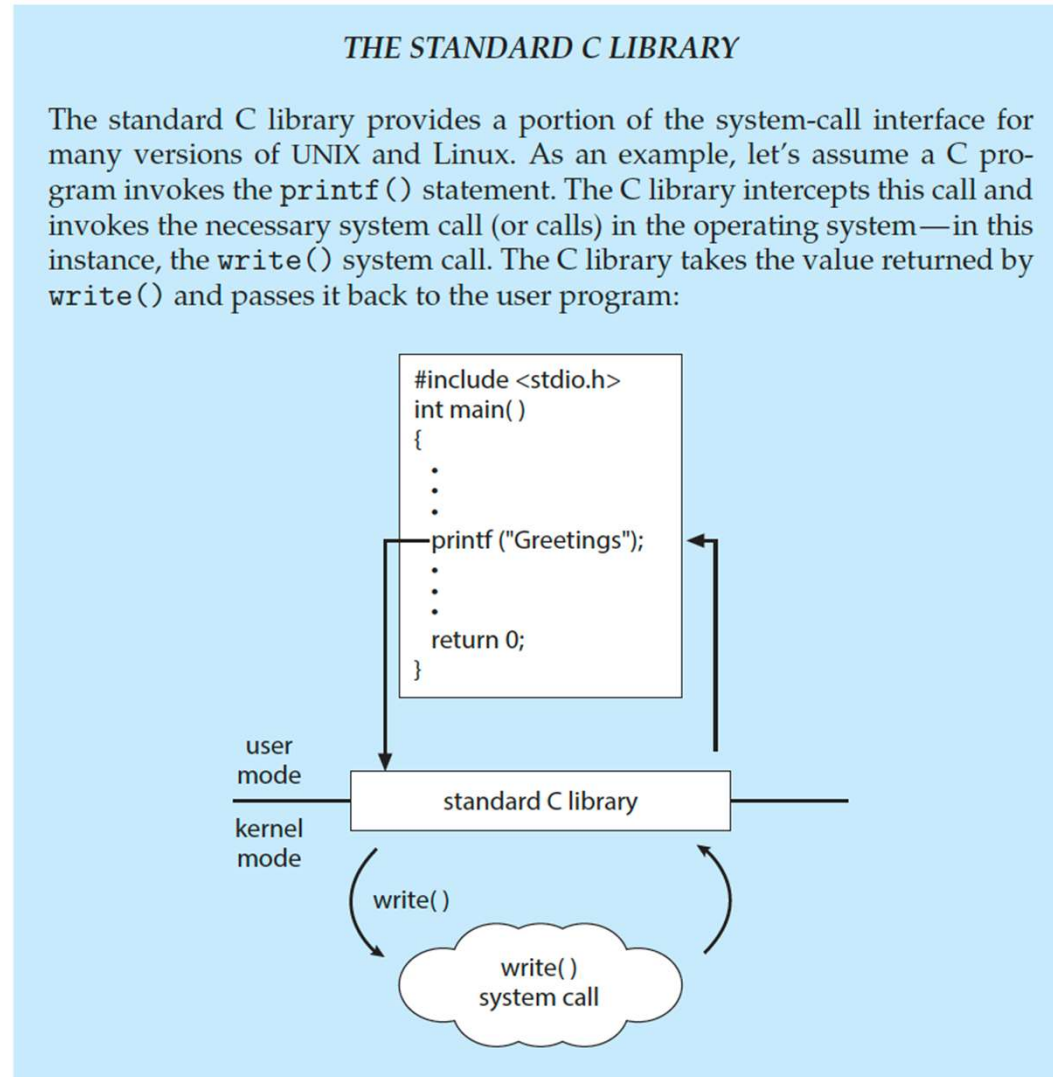
The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()



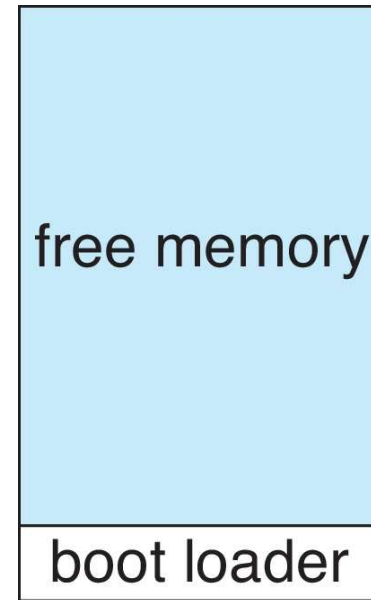
STANDARD C LIBRARY EXAMPLE

- C program invoking `printf()` library call, which calls `write()` system call



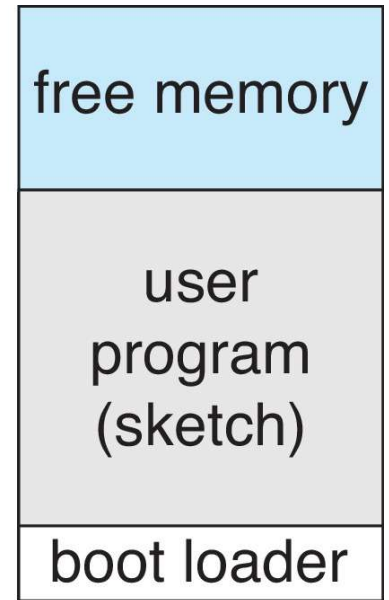
EXAMPLE: ARDUINO

- Single-tasking
- No operating system
- Programs (sketch) loaded via USB into flash memory
- Single memory space
- Boot loader loads program
- Program exit -> shell reloaded



(a)

At system startup



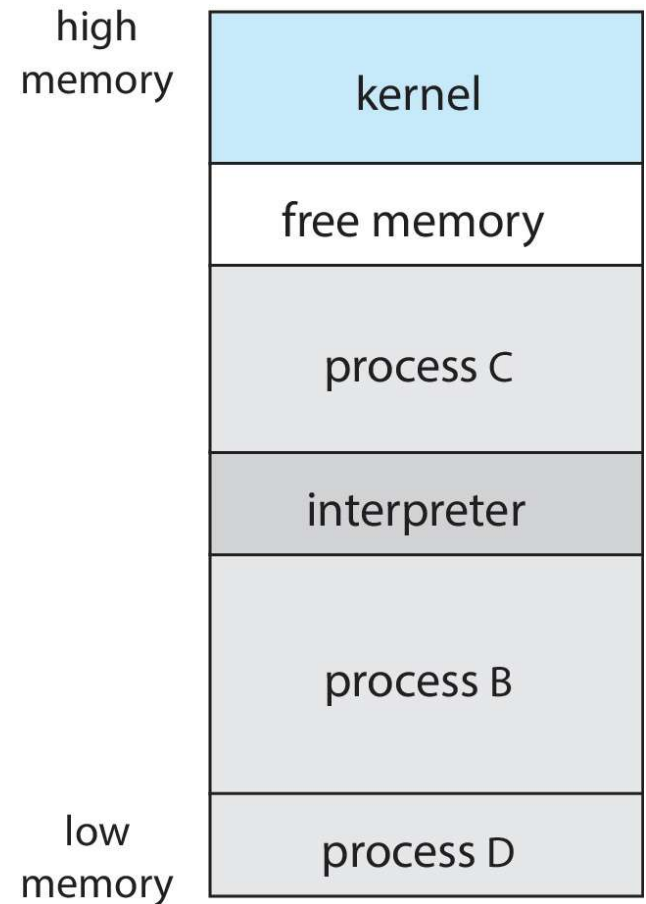
(b)

running a program



EXAMPLE: FREEBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes `fork()` system call to create process
 - Executes `exec()` to load program into memory
 - Shell waits for process to terminate or continues with user commands
- Process exits with:
 - code = 0 – no error
 - code > 0 – error code



INTERRUPTS



COMPUTER-SYSTEM OPERATION

- I/O devices and the CPU can execute concurrently
- Each **device controller** is **in charge of a particular device type** for example, a disk drive, audio device, or graphics display)
- Each **device controller has a local buffer storage** and a set of **special-purpose registers**.
- The device controller is **responsible for moving the data between the peripheral devices** that it controls and its local buffer storage.
- Each **device controller type** has an **operating system device driver** to manage it
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- **Device controller informs CPU** that it has finished its operation by causing an **interrupt**



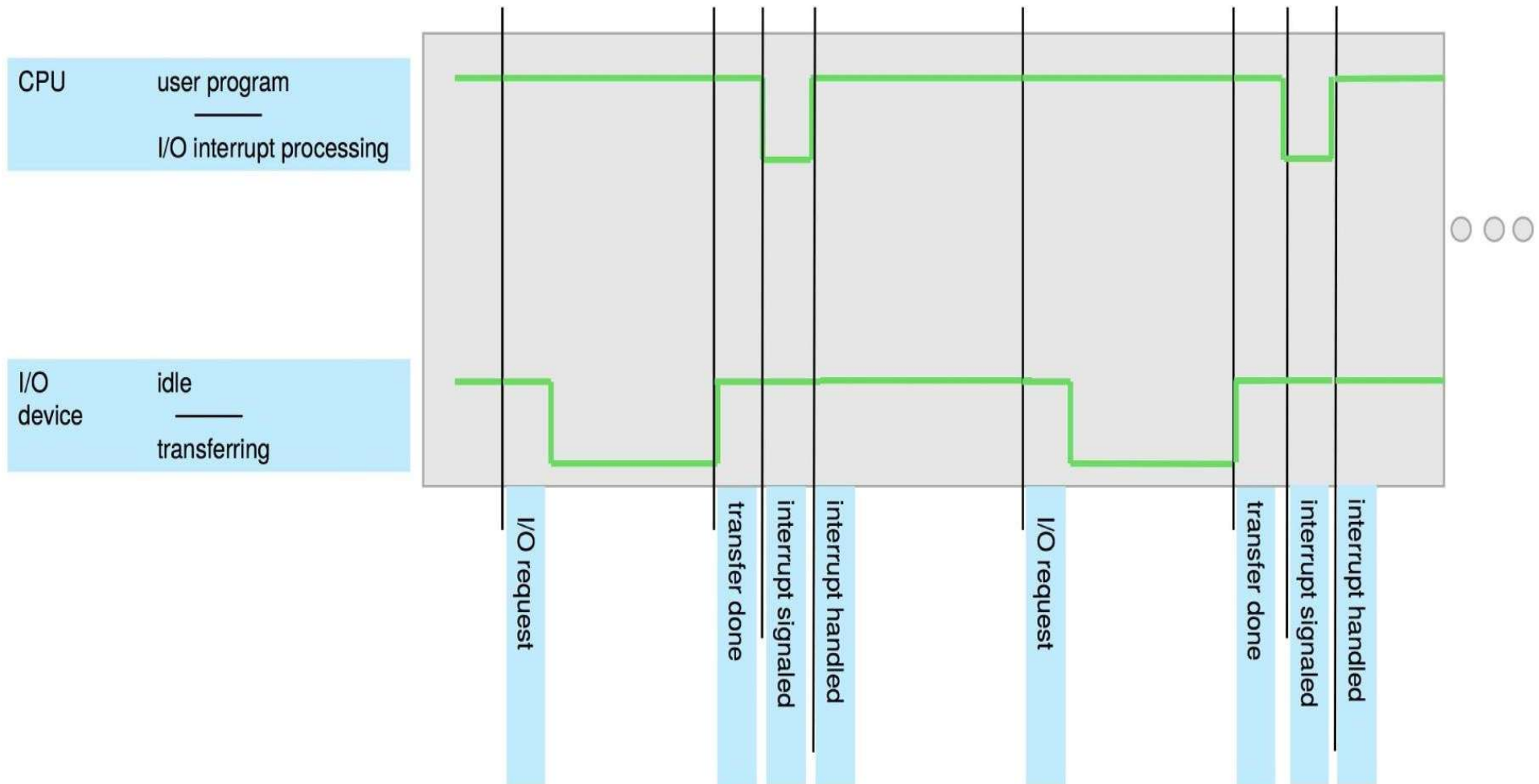
INTERRUPTS

- Interrupt is a signal sent to the CPU can be triggered by the hardware or software indicating that an event needs immediate attention.
- It temporarily halts the current program execution, saves the state, and transfers control to an interrupt service routine (ISR).
- That is, When the CPU is interrupted, it stops what it is doing and immediately transfers execution to a fixed location.
- Device controllers and hardware faults raise interrupts.
- The fixed location usually contains the starting address where the service routine for the interrupt is located.
- The interrupt service routine executes; on completion, the CPU resumes the interrupted computation.



INTERRUPTS

Interrupt timeline for a single program doing output

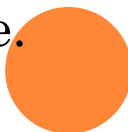


INTERRUPTS

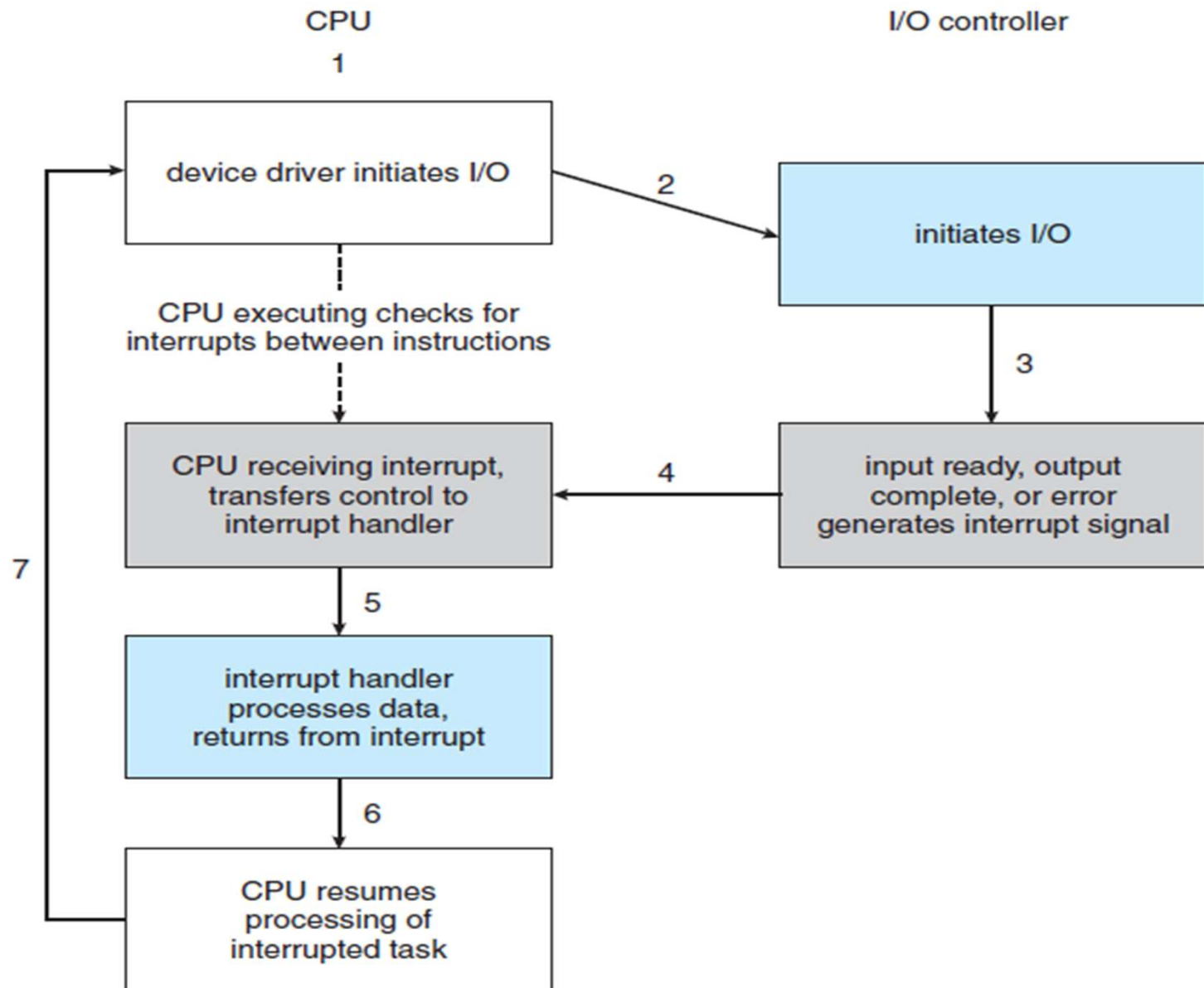
- The interrupt must **transfer control** to the **appropriate interrupt service routine** through the **interrupt vector**, which **contains the addresses** of all the **service routines**.
- Interrupts must be **handled quickly**.
- Interrupt architecture must **save the address of the interrupted instruction**.
- After the interrupt is serviced, the saved return address is loaded into the **program counter**, and the interrupted computation resumes as though the interrupt had not occurred.
- A **trap** or **exception** is a **software-generated interrupt** caused either by an **error or a user request**



INTERRUPT IMPLEMENTATION

- The **CPU hardware** has a **wire called the interrupt-request line** that the CPU senses after executing every instruction.
 - When the CPU detects that a controller has asserted a signal on the interrupt-request line, **it reads the interrupt number and jumps to the interrupt-handler routine** by using that interrupt number as an index into the interrupt vector.
 - It then **starts execution at the address associated with that index**.
 - The **interrupt handler saves the following states**:
 - determines the cause of the interrupt,
 - performs the necessary processing,
 - performs a state restore,
 - and executes a return from interrupt instruction to return the CPU to the execution state prior to the interrupt.
 - The device controller **raises** an interrupt by asserting a signal on the interrupt request line
 - The CPU **catches** the interrupt and **dispatches** it to the interrupt handler, and the handler **clears** the interrupt by servicing the device.
- 

Interrupt-driven I/O cycle



TYPES OF INTERRUPT

- **Maskable Interrupts**
 - CPU can disable or “mask” the interrupts using specific instructions.
- **Non Maskable Interrupts**
 - These interrupts cannot be disabled or ignored by the CPU.
- **Vectored Interrupts**
 - These interrupts reduce the need for a single interrupt handler to search all possible sources of interrupts to determine which one needs service.
 - a device requesting an interrupt identifies itself directly by sending a special code to the processor over the bus. This enables the processor to identify the device that generated the interrupt.
 - The special code can be the starting address of the ISR or where the ISR is located in memory and is called the interrupt vector.



TYPES OF INTERRUPT

- **Priority Interrupts**
 - This can be handled by **Interrupt Chaining**. Interrupt chaining is a technique where multiple devices share a single interrupt line, and the CPU identifies the source of the interrupt by passing control through a chain of devices in priority order.
 - These levels enable the CPU to **defer the handling of low-priority interrupts** without masking all interrupts and makes it possible for a high-priority interrupt to preempt the execution of a low-priority interrupt.



